

Операционные системы

Внешний курс. Введение в Linux. Продвинутые темы.

Кара Егор Андреевич

Содержание

1	Цель работы	8
2	Выполнение внешнего курса.	9
2.1	Нужно нажать ” : «, затем»q”, затем «Enter» на клавиатуре, чтобы выйти из редактора vim.	9
2.2	Создаем файл и вписываем туда «Strange_ TEXT is_here. 2=2 YES!». Только это утверждение верно: «После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w».	10
2.3	Из предложенных ниже наборов нажатий клавиш d2w\$bifour four , ddithree four four four five, d2wwyWPr выполняют такое редактирование.	10
2.4	Задача: «Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово Windows, на такие же строки, но со словом Linux. Если в какой-то строке слово Windows встречается больше, чем один раз, то заменить на Linux в этой строке нужно только самое первое из этих слов.»	11
2.5	Все верные утверждения, которые относятся к третьему режиму работы vim – режим выделения (Visual):	11
2.6	Если попробовать при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды только из набора C будут появляться.	12
2.7	Абсолютный путь до созданного файла file1.txt по окончании работы скрипта будет выглядеть как /home/bi/file1.txt, так как сначала мы оказываемся дома /home/bi, затем создаём file1.txt, затем уходим в ./Desktop.	13
2.8	Из представленных ниже строк __variable, VARiable, variable123, variable, variable_123 могут быть именами переменных в bash. Имена переменных всегда могут содержать латинские буквы (любого регистра), знак подчеркивания , и цифры (но при условии, что имя переменной не начинается с цифры)	13
2.9	Скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:	14
2.10	Условия, при которых echo напечатает на экран True вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные:	14

2.11 Сначала four, потом four фрагмент bash-скрипта выведет на экран, если сначала этот скрипт запустили задав переменную var=3, а затем запустили еще раз, но уже с var=5.	15
2.12 Скрипт на bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории.	16
2.13 Если запустить этот скрипт, то 5 раз «start» и 4 раза «finish» будут выведены на экран.	18
2.14 Скрипт на bash, который будет определять в какую возрастную группу попадают пользователи.	19
2.15 Предложенные ниже инструкции на скриншоте увеличат значение переменной a на значение переменной b:	21
2.16 Команда echo выведет на экран:	22
2.17 Все верные утверждения или правильно работающие конструкции if:	23
2.18 Строка, которая выведет на экран команду echo «counters are \$c1 and \$c2», если она находится в скрипте после десяти вызовов функции counter с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.	24
2.19 Скрипт на bash, который будет искать наибольший общий делитель (НОД, greatest common divisor, GCD) двух чисел.	25
2.20 Калькулятор на bash.	27
2.21 Все файлы, которые найдет команда find /home/bi -iname «star», но НЕ найдет команда find /home/bi -name "star":	28
2.22 Все верные утверждения из перечисленных ниже:	30
2.23 Все кроме file3 будут найдены по команде find /home/bi -mindepth 2 -maxdepth 3 -name "file*"	31
2.24 Если выполнить на этом файле команды, то results.txt будет одинакового размера во всех случаях.	32
2.25 На скриншоте варианты ответа, которые выведет на экран команда grep -E «[xklXKL]?[uU]buntu\$» text.txt.	33
2.26 Если в команде sed -n «/[a-z]*/p» text.txt не указывать опцию -n, то каждая строка будет выведена два раза.	34
2.27 Инструкция sed, которая заменит все «аббревиатуры» в файле input.txt на слово «abbreviation» и запишет результат в файл edited.txt	35
2.28 Опцию -p, -persist нужно указать при запуске gnuplot, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики.	37
2.29 Какое в этом случае будет название у построенного ряда данных и сколько будет нарисовано точек на графике? Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена).	38

2.30 Одна команда, которую нужно добавить в скрипт, для выполнения этой задачи.	39
2.31 Изменил инструкции в файле move.rot (т.е. добавлять и удалять инструкции нельзя!) таким образом, чтобы:	40
2.32 Команды, которые установят файлу file.txt права доступа rwxrw-r-, если изначально у него были права r-r-r-.	43
2.33 После выполнения команды sudo chown user:group dir, sudo chmod o+w dir, sudo chown user dir, sudo chmod a+w dir user из группы group всё-таки сможет создать файл внутри dir.	44
2.34 Характеристики файла, которые можно посчитать с использованием команды wc.	46
2.35 Команда, которая выведет сколько места на диске занимает текущая директория (при этом размер нужно вывести в удобном для чтения формате (например, вместо 2048 байт надо выводить 2.0К) и больше на экран выводить ничего не нужно)	47
2.36 Максимально короткая команда (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами dir1, dir2, dir3.	48

3 Вывод

49

Список иллюстраций

2.1	" : «, затем»q", затем «Enter»	9
2.2	После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w	10
2.3	d2w\$bifour four , ddithree four four four five, d2wwywPp	10
2.4	:%s/Windows/Linux/	11
2.5	Режим выделения (Visual)	12
2.6	Команды только из набора C будут появляться.	12
2.7	/home/bi/file1.txt	13
2.8	__variable, VARiable, variable123, _variable, variable_123	13
2.9	Скрипт на bash	14
2.10	Условия, при которых echo напечатает на экран True	15
2.11	Сначала four, потом four	16
2.12	Скрипт на bash	17
2.13	5 раз «start» и 4 раза «finish»	18
2.14	Вопрос	19
2.15	Ответ	20
2.16	Инструкции, которые увеличат значение переменной a на значе- ние переменной b	21
2.17	Команда echo	22
2.18	Все верные утверждения или правильно работающие конструк- ции if	23
2.19	Команда echo «counters are \$c1 and \$c2»	24
2.20	Условие задачи.	25
2.21	Ответ к задаче.	26
2.22	Условие задачи.	27
2.23	Ответ к задаче.	28
2.24	Все файлы, которые найдет команда find /home/bi -iname "star*"	29
2.25	Все верные утверждения	30
2.26	Все кроме file3	31
2.27	results.txt будет одинакового размера во всех случаях	32
2.28	Варианты ответа, которые выведет на экран команда grep -E «[xklXKL]?[uU]buntu\$» text.txt.	33
2.29	Каждая строка будет выведена два раза	34
2.30	Вопрос.	35
2.31	Ответ.	36
2.32	Опция -p, -persist	37

2.33 Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)	38
2.34 Вопрос.	39
2.35 Ответ	40
2.36 Вопрос.	41
2.37 Ответ	42
2.38 Команды, которые установят файлу file.txt права доступа rwxrw-r-, если изначально у него были права r-r-r-.	43
2.39 Вопрос.	44
2.40 Ответ	45
2.41 Характеристики файла, которые можно посчитать с использованием команды wc.	46
2.42 Команда, которая выведет сколько места на диске занимает текущая директория.	47
2.43 Максимально короткая команда	48

Список таблиц

1 Цель работы

Ознакомление с Linux на более продвинутом уровне, получение более сложных навыков работы с Linux.

2 Выполнение внешнего курса.

2.1 Нужно нажать ” : «, затем»q”, затем «Enter» на клавиатуре, чтобы выйти из редактора vim.

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

✓ Прекрасный ответ.

- ☒ " : ", затем "q", затем "Enter"
- ☐ "Q"
- ☐ "Esc"
- ☐ "Ctrl", затем "x"
- ☐ "q", затем "Enter"

Следующий шаг

Решить снова

Рисунок 2.1: ” : «, затем»q”, затем «Enter»

2.2 Создаем файл и вписываем туда «Strange_ TEXT is_here. 2=2 YES!». Только это утверждение верно: «После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w».

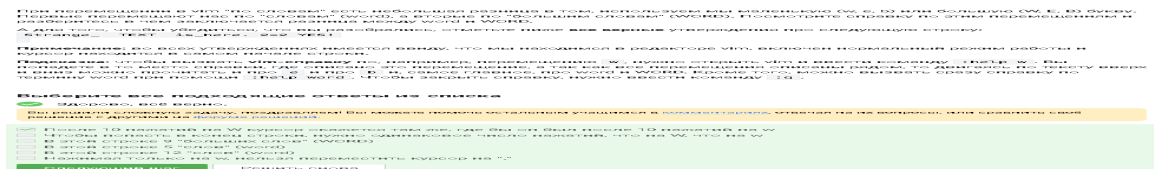


Рисунок 2.2: После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w

2.3 Из предложенных ниже наборов нажатий клавиш d2w\$bifour four , ddithree four four four five, d2wwwywPr выполняют такое редактирование.

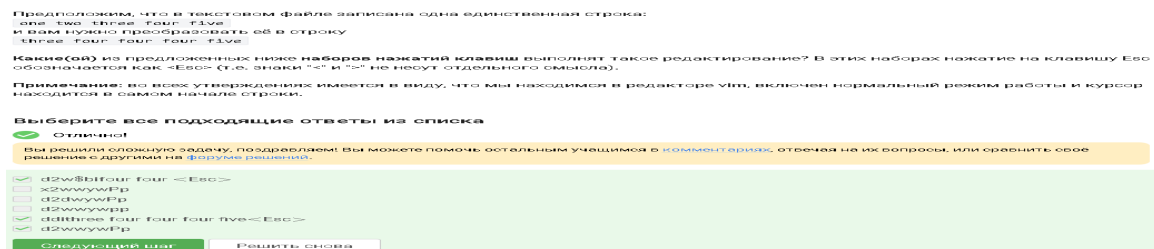


Рисунок 2.3: d2w\$bifour four , ddithree four four four five, d2wwwywPr

2.4 Задача: «Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово Windows, на такие же строки, но со словом Linux. Если в какой-то строке слово Windows встречается больше, чем один раз, то заменить на Linux в этой строке нужно только самое первое из этих слов.»

Для этого в vim нужно ввести `:%s/Windows/Linux/`

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно только самое первое из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. включая ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак не нужно.

Напишите текст

✓ Верно.

`:%s/Windows/Linux/`

Следующий шаг

Решить снова

Рисунок 2.4: `:%s/Windows/Linux/`

2.5 Все верные утверждения, которые относятся к третьему режиму работы vim – режим выделения (Visual):

Выйти из режима выделения можно, нажав клавишу Esc два раза.

Режим выделения открывается из нормального режима по нажатию «v».

Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –).

В режиме выделения можно использовать команды `d` (удалить) и `y` (скопировать).



Рисунок 2.5: Режим выделения (Visual)

2.6 Если попробовать при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды только из набора C будут появляться.

Каждый вызов любой программы (в том числе терминала) порождает новый процесс. 1ый `bash` является родительским процессом для вызванного `sh`, а `sh` является родителем 2ого процесса `bash`. Таким образом, запущено два процесса `bash`, и мы сейчас находимся во втором.

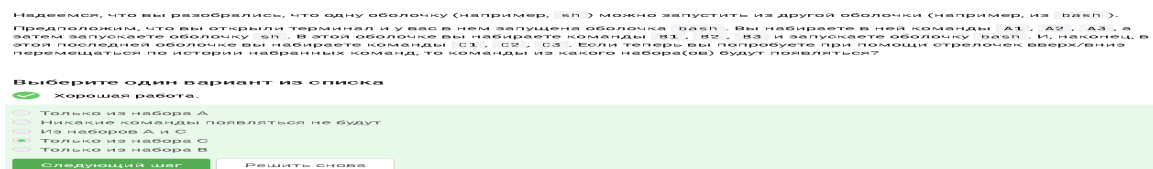


Рисунок 2.6: Команды только из набора C будут появляться.

2.7 Абсолютный путь до созданного файла file1.txt по окончании работы скрипта будет выглядеть как /home/bi/file1.txt, так как сначала мы оказываемся дома /home/bi, затем создаём file1.txt, затем уходим в ./Desktop.

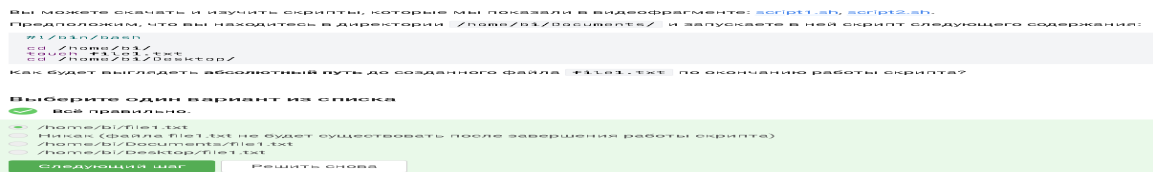


Рисунок 2.7: /home/bi/file1.txt

2.8 Из представленных ниже строк __variable, VARiable, variable123, variable, variable_123 могут быть именами переменных в bash. Имена переменных всегда могут содержать латинские буквы (любого регистра), знак подчеркивания , и цифры (но при условии, что имя переменной не начинается с цифры)

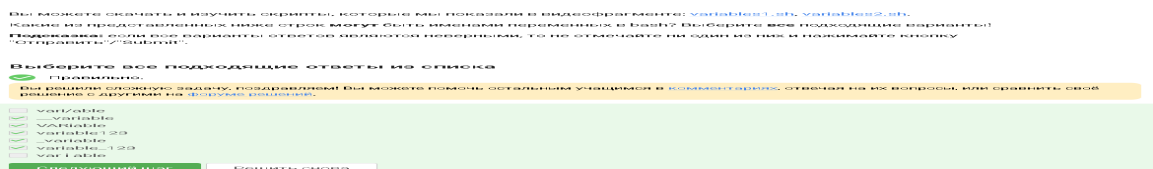


Рисунок 2.8: __variable, VARiable, variable123, _variable, variable_123

2.9 Скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
var1=$1
var2=$2
echo «Arguments are: $1=var1$2 =var2»
```

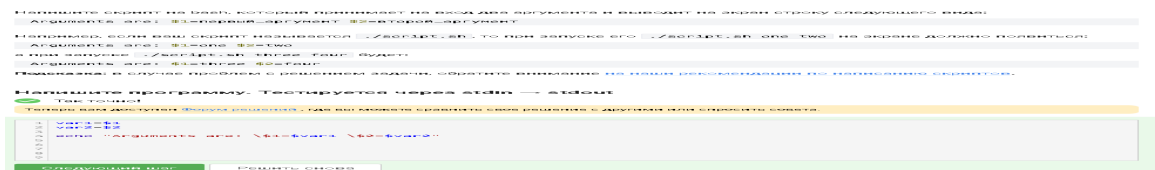


Рисунок 2.9: Скрипт на bash

2.10 Условия, при которых echo напечатает на экран True вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные:

```
$var1 == $var2 || var1 != var2
$#-ge 0
-e $0
5 -ge 5
-z “ ”
```

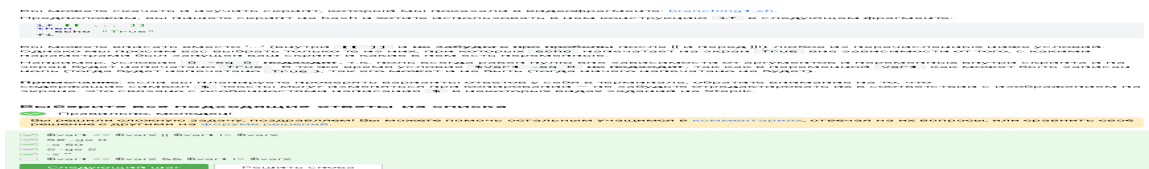


Рисунок 2.10: Условия, при которых echo напечатает на экран True

2.11 Сначала four, потом four фрагмент bash-скрипта выведет на экран, если сначала этот скрипт запустили задав переменную var=3, а затем запустили еще раз, но уже с var=5.

2.11.1 Анализ поведения

Случай 1: Скрипт запускается с var=3

1. var становится 3.
2. `if [[ 3 -gt 5 ]]` -> false (3 не больше 5).
3. `elif [[ 3 -lt 3 ]]` -> false (3 не меньше 3).
4. `elif [[ 3 -eq 4 ]]` -> false (3 не равно 4).
5. Выполняется блок `else`.
6. `echo "four"` выводится на экран.

Случай 2: Скрипт запускается с var=5

1. var становится 5.
2. `if [[ 5 -gt 5 ]]` -> false (5 не больше 5).
3. `elif [[ 5 -lt 3 ]]` -> false (5 не меньше 3).
4. `elif [[ 5 -eq 4 ]]` -> false (5 не равно 4).
5. Выполняется блок `else`.
6. `echo "four"` выводится на экран.

2.11.2 Вывод

В обоих случаях скрипт выводит строку «four».

Таким образом, правильный вариант:

Сначала four, потом four

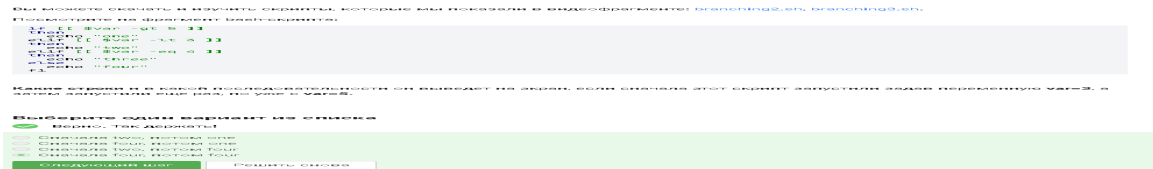


Рисунок 2.11: Сначала four, потом four

2.12 Скрипт на bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории.

```
if [[ $1 -eq 1 ]]; then echo «$1 student» elif [[ $1 -gt 1 && $1 -le 4 ]]; then echo «$1 students» elif [[ $1 -ge 5 ]]; then echo «A lot of students» else echo «No students» fi
```


Напишите скрипт на `bash`, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): вывести нужно только строку справа, т.е. "`-->`" выводить не нужно.

Примечание б): в последней строке слова "lot" с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно проверять вход (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать не нужно!

Пример №21: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №22: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через `stdin → stdout`

✓ Верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 if [[ $1 -eq 1 ]]; then
2   echo "1 student"
3 elif [[ $1 -gt 1 && $1 -le 4 ]]; then
4   echo "$1 students"
5 elif [[ $1 -ge 5 ]]; then
6   echo "A lot of students"
7 else
8   echo "No students"
9 fi
10
11
12
13
14
```

Следующий шаг

Решить снова

Рисунок 2.12: Скрипт на `bash`

2.13 Если запустить этот скрипт, то 5 раз «start» и 4 раза «finish» будут выведены на экран.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово "start", а сколько раз слово "finish"?

Выберите один вариант из списка

☒ Отлично!

- ☐ 5 раз "start" и 2 раза "finish"
- ☐ 5 раз "start" и ни разу "finish"
- ☐ 3 раза "start" и 3 раза "finish"
- ☒ 5 раз "start" и 4 раза "finish"

Следующий шаг

Решить снова

Рисунок 2.13: 5 раз «start» и 4 раза «finish»

2.14 Скрипт на bash, который будет определять в какую возрастную группу попадают пользователи.

Напишите скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение 'enter your name:' и ждать от пользователя ввода имени (используйте `read`, чтобы прочесть его). Когда имя введено, то скрипт должен написать 'enter your age:' и ждать ввода возраста (опять нужен `read`). Когда возраст введен, скрипт пишет на экран '<Имя>, your group is <группа>', где <группа> определяется на основе возраста по следующим правилам:

- младше либо равно 16: 'child',
- от 17 до 25 (включительно): 'youth',
- старше 25: 'adult'.

После этого скрипт опять выводит сообщение 'enter your name:' и всё начинается по новой (бесконечный цикл!). Если в какой-то момент работы скрипта будет введено пустое имя или возраст 0, то скрипт должен написать на экран 'bye' и закончить свою работу (выход из цикла).

Примеры корректной работы скрипта:

№1

```
./script.sh
enter your name:
Egor
enter your age:
16
Egor, your group is child
enter your name:
Elena
enter your age:
0
bye
```

№2:

```
./script.sh
enter your name:
Elena Petrovna
enter your age:
25
Elena Petrovna, your group is youth
enter your name:

bye
```

Рисунок 2.14: Вопрос

Напишите программу. Тестируется через stdin → stdout

✓ Верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

```
1 while [[ 1==1 ]]
2 do
3     group=""
4     echo "enter your name:"
5     read name
6     if [[ -z $name ]]
7     then
8         break
9     fi
10    echo "enter your age:"
11    read age
12    if [[ $age -eq 0 ]]
13    then
14        break
15    fi
16    if [[ $age -le 16 ]]
17    then
18        group="child"
19    elif [[ $age -le 25 ]]
20    then
21        group="youth"
22    else
23        group="adult"
24    fi
25    echo "$name, your group is $group"
26 done
27 echo "bye"
```

Следующий шаг

Решить снова

Рисунок 2.15: Ответ

2.15 Предложенные ниже инструкции на скриншоте увеличат значение переменной `a` на значение переменной `b`:

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b`? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.

Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ `let "a+=b"`
☒ `let a=$a+$b`
☒ `let "a=$a+$b"`
☒ `let "a = a + b"`
☒ `let "a+=b"`

Следующий шаг

Решить снова

Рисунок 2.16: Инструкции, которые увеличат значение переменной `a` на значение переменной `b`

2.16 Команда echo выведет на экран:

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: programs.sh.

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

☒ Хорошая работа.

- ☐ pwd
- ☐ `pwd`
- ☐ Код возврата команды pwd (0 в случае успешного выполнения и не 0 в случае ошибок)
- ☒ /home/bi
- ☐ /home/bi/Documents

Следующий шаг

Решить снова

Рисунок 2.17: Команда echo

2.17 Все верные утверждения или правильно работающие конструкции if:

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполнятся, если программа закончилась с кодом 0). Однако это **не всегда правда!** Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно **косая кавычка** (```), а не обычная (`"`) или двойная (`'`).

Выберите все подходящие ответы из списка

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Ничего сделать нельзя
- ☒ `if `program > some_file.txt``
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`
- ☐ `if [[ `program` -eq 0 ]]`
- ☐ Сначала `var=`program``, затем `if [[ $var -eq 0 ]]`

Следующий шаг

Решить снова

Рисунок 2.18: Все верные утверждения или правильно работающие конструкции `if`

2.18 Строка, которая выведет на экран команду echo «counters are \$c1 and \$c2», если она находится в скрипте после десяти вызовов функции counter с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Объяснение: первая переменная локальная => пустая строка, вторая сумма арифметической прогрессии от 1 до 10 (55), но еще умноженная на 2 => 110

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
    local let "c1+=1"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже строку, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте после десяти вызовов функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 😊). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст

✓ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Следующий шаг

Решить снова

Рисунок 2.19: Команда echo «counters are \$c1 and \$c2»

2.19 Скрипт на bash, который будет искать наибольший общий делитель (НОД, greatest common divisor, GCD) двух чисел.

Напишите программу. Тестируется через stdin → stdout

✓ Верно. Так держать!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 # put your shell (bash) code here
2 while [ true ]
3 do
4     read n1 n2
5     if [ -z $n1 ]; then
6         echo "bye"
7         break
8     else
9         gcd () {
10             remainder=1
11             if [ $n2 -eq 0 ]
12             then
13                 echo "bye"
14             fi
15             while [ $remainder -ne 0 ]
16             do
17                 remainder=$((n1%n2))
18                 n1=$n2
19                 n2=$remainder
20             done
21         }
22         gcd $1 $2
23         echo "GCD is $n1"
24     fi
25 done
26
27
28
29
```

Следующий шаг

Решить снова

Рисунок 2.20: Условие задачи.

Напишите скрипт на `bash`, который будет искать наибольший общий делитель ([НОД](#), greatest common divisor, GCD) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждет ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение "GCD is <посчитанное значение>", например, для чисел 15 и 25 это будет "GCD is 5". После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран "bye" и закончить свою работу.

Вычисление НОД не сложно реализовать с помощью [алгоритма Евклида](#). Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их `M` и `N`). Если аргументы равны, то мы нашли НОД – он равен `M` (или `N`), нужно выводить соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если `M` больше `N`, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем `(M-N)`, а в качестве второго `N`. Если же наоборот, `M` меньше `N`, то запускаем функцию `gcd` с первым аргументом `M`, а вторым `(N-M)`.

Пример корректной работы скрипта:

```
./script.sh
10 15
gcd is 5
7 3
gcd is 1
bye
```

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd` !

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Рисунок 2.21: Ответ к задаче.

2.20 Калькулятор на bash.

Напишите программу. Тестируется через stdin → stdout

✓ Абсолютно точно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 # put your shell (bash) code here
2 #!/bin/bash
3 while [[ True ]]
4 do
5     read birinchi amal ikkinchi
6     if [[ $birinchi == "exit" ]]
7     then
8         echo "bye"
9         break
10    elif [[ "$birinchi" =~ "^[-0-9]+$" && "$ikkinchi" =~ "^[-0-9]+$" ]]
11    then
12        echo "error"
13        break
14    else
15        case $amal in
16            "+") let "result = birinchi + ikkinchi";;
17            "-") let "result = birinchi - ikkinchi";;
18            "/" ) let "result = birinchi / ikkinchi";;
19            "*" ) let "result = birinchi * ikkinchi";;
20            "%" ) let "result = birinchi % ikkinchi";;
21            "**") let "result = birinchi ** ikkinchi";;
22            *) echo "error" ; break ;;
23        esac
24        echo "$result"
25    fi
26 done
27
28
29
30
```

Следующий шаг

Решить снова

Рисунок 2.22: Условие задачи.

Напишите **калькулятор** на **bash**. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово **'exit'**. В этом случае скрипт должен вывести на экран слово **'bye'** и завершить работу.
2. Три аргумента через пробел – первый операнд (целое число), операция (одна из **'+', '-', '*', '/', '%', '**'**) и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого переходим в режим ожидания новой команды.
3. Любая другая команда из одного аргумента или из трех аргументов, но с операцией не из списка. В этом случае нужно вывести на экран слово **'error'** и завершить работу.

Чтобы проверить работу скрипта, вы можете записать сразу несколько команд в файл и передать его скрипту на **stdin** (т.е. выполнить **./script.sh < input.txt**). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
10 + 1
2 ** 10
exit
```

то на экране будет:

```
11
1024
bye
```

Если же на вход поступит следующий файл:

```
3 - 5
2/10
exit
```

то на экране будет:

```
-2
error
```

т.к. вторая команда была **некорректной** (в ней всего один аргумент, т.к. нет пробелов между числами и операцией, а единственная допустимая команда из одного аргумента это **'exit'**).

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Рисунок 2.23: Ответ к задаче.

2.21 Все файлы, которые найдет команда **find**

/home/bi -iname «star», но НЕ найдет команда find

/home/bi -name "star»:

Star_Wars.avi

STARS.txt

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek_OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ** найдет команда `find /home/bi -name "star*" ?`

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

- ☒ `Star_Wars.avi`
- ☐ `star_trek_OST.mp3`
- ☐ `stardust.mpeg`
- ☒ `STARS.txt`
- ☐ `Eddard_Stark_biography.txt`

Следующий шаг

Решить снова

Рисунок 2.24: Все файлы, которые найдет команда `find /home/bi -iname "star*"`

2.22 Все верные утверждения из перечисленных ниже:

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте все верные утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным
- ☒ Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же
- ☐ Опции `-path` и `-name` всегда работают одинаково
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`

Следующий шаг

Решить снова

Рисунок 2.25: Все верные утверждения

2.23 Все кроме file3 будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*"`

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/  
├── dir1  
│   ├── file1  
│   └── dir2  
│       ├── file2  
│       └── dir3  
│           └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

☒ Прекрасный ответ.

- ☐ Ни один файл найден не будет
- ☐ Только file1
- ☒ Все кроме file3
- ☐ Только file2
- ☐ Все кроме file1

Следующий шаг

Решить снова

Рисунок 2.26: Все кроме file3

2.24 Если выполнить на этом файле команды, то results.txt будет одинакового размера во всех случаях.

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в каждой строке есть слово `"word"`. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

☒ Хорошие новости, верно!

- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt`
- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях
- ☐ `grep -C 1 "word" file.txt > results.txt`

Следующий шаг

Решить снова

Рисунок 2.27: `results.txt` будет одинакового размера во всех случаях

2.25 На скриншоте варианты ответа, которые выведет на экран команда `grep -E «[xklXKL]?[uU]buntu$» text.txt`.

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xklXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

☒ Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04
- ☐ Lubuntu is better than Windows
- ☐ Uuuubuntu!
- ☒ Hmm, XKLubuntu
- ☒ Lubuntu is better than Ubuntu
- ☒ Linux is not always Ubuntu

Следующий шаг

Решить снова

Рисунок 2.28: Варианты ответа, которые выведет на экран команда `grep -E «[xklXKL]?[uU]buntu$» text.txt`.

2.26 Если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n`, то каждая строка будет выведена два раза.

Что произойдет, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n`?

Выберите один вариант из списка

☒ Отличное решение!

- ☒ Каждая строка будет выведена два раза
- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита
- ☐ На экран будет выведено всё содержимое файла `text.txt`
- ☐ Появится сообщение об ошибке

Следующий шаг

Решить снова

Рисунок 2.29: Каждая строка будет выведена два раза

2.27 Инструкция sed, которая заменит все «аббревиатуры» в файле input.txt на слово «abbreviation» и запишет результат в файл edited.txt

Запишите в форму ниже инструкцию `sed`, которая заменит все "аббревиатуры" в файле `input.txt` на слово "abbreviation" и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что в инструкции должны быть указаны и сам `sed`, и оба файла!

Под "аббревиатурой" будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте **не может быть две "аббревиатуры" подряд**. Например, текст " YOU YOU and YOU!" является **некорректным** (в нем есть две "аббревиатуры", но они идут подряд) и на таких примерах мы проверять вашу инструкцию **не будем**.

Пример: если у вас был текст "Hi, I heard these songs by ABBA, TLA and DM !", то он должен быть преобразован в "Hi, I heard these songs by ABBA, abbreviation and abbreviation !".

Примечание: после вашей замены "аббревиатуры" на слово "abbreviation" **количество пробелов** в тексте **не должно меняться!**

Внимание! Во время проверки **мы не запускаем команду**, которую вы ввели на реальном файле с "аббревиатурами" (это небезопасно, можно же ввести `rm -rf /*`)! Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её смысловую часть** (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах. К сожалению, наш запуск **не идеально повторяет sed**, но он очень близок к нему. Главная "несовместимость" заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. *, +, ? и {}). Однако эту "несовместимость" легко исправить указав при помощи "(" и ")" какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве "a") нужно записать как `(a+)?` (при этом запись `(a)+?`, конечно же, не поможет).

Рисунок 2.30: Вопрос.

Напишите текст



Верно. Так держать!

```
sed -r 's/ [A-Z]{2,} / abbreviation /g' input.txt > edited.txt
```

Следующий шаг

Решить снова

Рисунок 2.31: Ответ.

2.28 Опцию -p, --persist нужно указать при запуске **gnuplot**, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики.

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

☒ Хорошая работа.

- ☐ -s, --show-plots-after-exit
- ☒ -p, --persist
- ☐ Такой опции не существует
- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!

Следующий шаг

Решить снова

Рисунок 2.32: Опция -p, --persist

2.29 Какое в этом случае будет название у построенного ряда данных и сколько будет нарисовано точек на графике? Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена).

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет название у построенного ряда данных и сколько будет нарисовано точек на графике?

Выберите один вариант из списка

☒ Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Название -- первое значение из первого столбца, нарисовано 10 точек
- ☐ Название "noname", нарисовано 10 точек
- ☐ Название "data.csv' using 1:2", нарисовано 10 точек
- ☐ Название -- первое значение из второго столбца, нарисовано 10 точек
- ☒ Название -- первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)

Следующий шаг

Решить снова

Рисунок 2.33: Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)

2.30 Одна команда, которую нужно добавить в скрипт, для выполнения этой задачи.

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предположим, что вы пишете gnuplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде "point <номер точки>, value <значение соответствующей переменной>". Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20. Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно**!

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин *конкатенация*, который важен для выполнения данного задания. Под *конкатенацией* обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Рисунок 2.34: Вопрос.

Напишите текст

✓ Абсолютно точно.

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)
```

Следующий шаг

Решить снова

Рисунок 2.35: Ответ

2.31 Изменил инструкции в файле `move.rot` (т.е. добавлять и удалять инструкции нельзя!) таким образом, чтобы:

График отразился зеркально относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте. Изображение стало вращаться в обратную сторону. То есть если раньше вращалось «влево», то теперь станет «вправо». Вращение стало в два раза быстрее. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. добавлять и удалять инструкции нельзя!) таким образом, чтобы:

- График отразился зеркально относительно горизонтальной поверхности. То есть там, где была точка $(10, 10, 200)$, станет точка $(10, 10, -200)$, где была точка $(-10, -10, 200)$ станет $(-10, -10, -200)$ и т.д. При этом точка $(0, 0, 0)$ останется на месте.
- Изображение стало вращаться в обратную сторону. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало в два раза быстрее. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки не может запустить на вашем файле `move.rot` программу `gnuplot` и сравнить полученный график с заданным. Вместо этого мы анализируем команды, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в `gnuplot` работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Рисунок 2.36: Вопрос.

Напишите текст

✓ Верно.

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Следующий шаг

Решить снова

Рисунок 2.37: Ответ

2.32 Команды, которые установят файлу file.txt права доступа rwxrw-r--, если изначально у него были права r-r-r--.

Какая команда(ы) установят файлу `file.txt` права доступа `rwxrw-r--`, если изначально у него были права `r--r-r--`. Укажите все верные варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнились все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите все подходящие ответы из списка

✓ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `chmod u-wx file.txt; chmod g-w file.txt`
- ☒ `chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt`
- ☒ `chmod 764 file.txt`
- ☒ `chmod ug+w file.txt; chmod u+x file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☐ `chmod rwxrw-r-- file.txt`

Следующий шаг

Решить снова

Рисунок 2.38: Команды, которые установят файлу `file.txt` права доступа `rwxrw-r--`, если изначально у него были права `r-r-r--`.

2.33 После выполнения команды `sudo chown`

`user:group dir, sudo chmod o+w dir, sudo chown`

`user dir, sudo chmod a+w dir` из группы `group`

всё-таки сможет создать файл внутри `dir`.

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `gwxg-xg-x` (владелец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды `user` из группы `group` всё-таки сможет создать файл внутри `dir`? Укажите **все верные** варианты ответов!

Примечание: считаем, что все команды выполняются от имени `user`, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно.

Дело в том, что если создать при помощи `sudo` файл с правами `gw-g--g--` в директории, которая принадлежит пользователю, то возникнет любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено удалять **все** файлы внутри его директории) и может прочитать его содержимое (т.к. право `"r"` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `"w"` у файла есть только для `root`). При этом некоторые "умные" редакторы, например, `vim` позволят даже редактировать этот файл, но сделают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалять мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `gw-g--g--`, пользователь может сделать с этим файлом почти всё что угодно!

В случае же, когда речь идет о директории созданной `root`, ситуация будет проще: пользователь сможет посмотреть её содержимое (у него есть право `"r"`), но удалять и создавать файлы в ней не сможет (права `"w"` у него нет).

Важно отметить, что *директории в Linux* это в каком-то смысле *файлы*. Содержимое такого "файла" — это записи о файлах и поддиректориях этой директории (грубо говоря их *названия*). Таким образом, право `"r"` у директории дает возможность просматривать "записи", т.е. просматривать её состав. Право `"w"` у директории дает возможность удалять/добавлять новые "записи", т.е. удалять/создавать файлы/поддиректории в ней.

На самом деле и это еще не всё. Существует так называемый [sticky bit](#) (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса **Alexey Antipovsky** за помощь в оформлении **Примечания 2!**

Рисунок 2.39: Вопрос.

Выберите все подходящие ответы из списка

✓ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☒ sudo chown user:group dir

☒ sudo chmod o+w dir

☐ chown user:group dir

☒ sudo chown user dir

☐ sudo chmod o+x dir

☒ sudo chmod a+w dir

Следующий шаг

Решить снова

Рисунок 2.40: Ответ

2.34 Характеристики файла, которые можно посчитать с использованием команды `wc`.

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Количество предложений
- ☒ Количество символов
- ☒ Количество строк
- ☒ Количество слов
- ☐ Количество определенных букв (например, количество букв "А")

Следующий шаг

Решить снова

Рисунок 2.41: Характеристики файла, которые можно посчитать с использованием команды `wc`.

2.35 Команда, которая выведет сколько места на диске занимает текущая директория (при этом размер нужно вывести в удобном для чтения формате (например, вместо 2048 байт надо вывести 2.0K) и больше на экран выводить ничего не нужно)

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом размер нужно вывести в удобном для чтения формате (например, вместо 2048 байт надо вывести 2.0K) и больше на экран выводить ничего не нужно). В команде указывайте только необходимые для выполнения задания опции и аргументы, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4M (также на экране может быть выведен еще и символ ".", обозначающий, что это размер именно текущей директории).

Напишите текст

✓ Хорошие новости, верно!

```
du -h -s
```

Следующий шаг

Решить снова

Рисунок 2.42: Команда, которая выведет сколько места на диске занимает текущая директория.

2.36 Максимально короткая команда (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами dir1, dir2, dir3.

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✓ Так точно!

```
mkdir dir{1..3}
```

Следующий шаг

Решить снова

Рисунок 2.43: Максимально короткая команда

3 Вывод

Ознакомились с Linux на более продвинутом уровне, получили более сложные навыки работы с Linux.